



Test corpus: what we validate against, and what we deliberately do not

The parsers in this repo are only as trustworthy as the input they have met. This records where that input comes from, and one thing we decided **not** to use.

What we use

1. Reference archives from SngCli

`internal/sng/testdata/reference-sngcli-v0.3.0.sng` was produced by **SngCli v0.3.0** (MIT, `mdsitton/SngFileFormat`) from a song folder we wrote. It is the only fixture that checks the reader against something other than our own understanding of the format — the fixtures in `mask_test.go` and `read_test.go` are built by an encoder written from the same reading of the spec as the reader, and two components sharing one misreading agree with each other perfectly.

SngCli is installed at `%LOCALAPPDATA%\Programs\sngcli\win-x64\SngCli.exe`. To regenerate:

```
SngCli.exe encode -i <folder-of-song-folders> -o <out> --noStatusBar -t 1
SngCli.exe decode -i <folder-of-sng-files> -o <out> --noStatusBar -t 1
```

`.gitignore` excludes `*.sng` so nobody commits a song library, with an explicit exception for `internal/sng/testdata/`.

2. YARG itself, as the end-to-end oracle

YARG v0.15.0 is installed portable at `%LOCALAPPDATA%\Programs\YARG\YARG.exe` — the official Windows x64 zip, no installer, no elevation, deletable in one command. LGPL-3.0-or-later, free, and downloaded from YARG's own release page.

This is the gate that matters for the writer: put a repacked `.sng` in a songs folder, let YARG scan it, and confirm it appears with the right metadata and is playable. Nothing we can assert about our own output substitutes for the client accepting it.

(YARC publishes a `.sig` beside each release asset but no plain checksum, and no public key we could find, so the download is recorded by its SHA-256 rather than verified:

`a2eac765a190e34a8acbda5b1351844704b427349d6d36b8cf1096f575055e41`.)

3. Charts we author

Hand-written `song.ini` and chart files exercising the messy cases: BOMs, Latin-1 bytes, duplicate keys, missing section headers, values containing `=`, free-text years, unknown keys, absurd numbers. These are in the unit tests.

What we deliberately do not use

YARG's Official Setlist

It would be the obvious corpus — real charts, real metadata, distributed by YARC themselves — and we are **not** using it.

`YARC-Official/Official-Setlist-Public` states plainly that the setlist is proprietary, that "musicians gave permission to use these songs in YARG specifically", that everything "**must** be used exclusively for YARG", and that users should not download it manually. Using it as the test corpus for a third-party song server is outside that permission — and serving it from one would be squarely outside it.

That is not a close call, and it matters more here than it would elsewhere: this project's own goal is to make charts distributable *separately* from music precisely to avoid copyright problems. A project with that goal cannot be casual about the provenance of its own test data.

The same caution applies to bulk-downloading community charts from Chorus Encore or similar: individual charters' licensing varies, and most community packages bundle commercial audio. Individual charts may be used where the charter's own licence permits it — that is a per-chart question, answered before the chart is added, not a bulk decision.

The gap this leaves, stated honestly

Self-authored input cannot produce unknown unknowns. We can write a `song.ini` with the mistakes we thought of; we cannot write one with the mistakes we did not. Two things partly close that gap:

- SngCli's output already taught us two things synthetic input never would — that duplicate keys resolve last-wins in the reference tool, and that a contained file's extension can lie about its container.
- YARG scanning our output is a real oracle, because it is the actual client with the actual parser.

Neither is a substitute for a large real library. If a legitimately-licensed corpus becomes available, revisit this.